

ES6 (ES2015)

1) let vs const vs var: List the key differences in scoping, redeclaration, and reassignment. When would you still reach for var (if ever)?

1. var ניתן להכריז עליו כמה פעמים שרוצים אפשר לעשות את זה לפני שמשתמשים בו אבל אם נכריז עליו בשורה 8 אז בשורות 1-7 ערכו יהיה undefined. אפשר גם לעשות לו השמה מחדש. var שהוכרז בתוך בלוק מוכר גם מחוץ לבלוק.

let ניתן להכריז עליו פעם אחת. אי אפשר להשתמש בו במשתנה שהוגדר על ידו לפני שהוכרז, נסיון לעשות כך יגרום להודאת שגיאה. ניתן לעשות לו השמה עם ערך חדש. אם הוא מוגדר בתוך בלוק הוא לא מוכר מחוץ לבלוק.

const נניתן להכריז עליו פעם אחת. אי אפשר להשתמש בו במשתנה שהוגדר על ידו לפני שהוכרז, נסיון לעשות כך יגרום להודאת שגיאה. מרגע שהוכרז אי אפשר לעשות לו השמה מחדש. אם הוא מוגדר בבלוק הוא לא מוכר מחוץ לבלוק.

מאז ש- const ו- let קיימים אין סיבה להשתמש ב- var גם לא כדאי להשתמש בו. בגלל החופשיות שבו ניתן להכריז עליו ולהגדיר אותו מחדש. יש לי פחות ביקורת עליו ואני עלול לעשות טעויות שיגרמו לי לשנות ערכים שלא רציתי במיוחד בקודים ארוכים ולא מוכרים.

2) Temporal Dead Zone (TDZ): What happens in each case and why?

```
console.log(a); // ?
let a = 10;
function f() {
  console.log(b); // ?
  const b = 5;
}
f();
```

2. במקרה הראשון יש הודעה שגיאה כי ניסינו להשתמש במשתנה שהוכרז ע"י let לפני שהכרזנו עליו.

במקרה השני נקבל הודעת שגיאה בגלל שהפונקציה צריכה להדפיס את b שמוכרז עם const רק שורה אחרי השורה שבה היא מנסה להדפיס (בדיוק כמו במקרה הראשון).

3) Block scope: Why does `{ let x = 1 } console.log(x)` throw but `{ var y = 2 } console.log(y)` works? Explain using scope rules.

3. x הוגדר עם let בתוך בלוק לכן הוא לא מוכר מחוץ לבלוק וכשמנסים להדפיס יש שגיאה. y גם הוגדר בתוך בלוק אבל הוגדר עם var ומה שמוגדר עם var בתוך בלוק מוכר גם מחוץ לבלוק ולכן זה עובד.

ES6 (ES2015)

4) Arrow functions and `this`: What does this print and why?

```
const obj = {  
  x: 42,  
  m() {  
    const inner = () => this.x;  
    return inner();  
  }  
};  
console.log(obj.m());
```

4. זה ידפיס 42 ה-`this` של המטודה `m` שהוגדרה כפונקציה רגילה מתייחס לאובייקט אליו היא שייכת ובאובייקט `x` הוא 42. הפונקציה `inner` הוגדרה כפונקציית `arrow` בתוך המטודה `m` ה-`this` שלה מתייחס ל-`this` של הסיבה בה היא הוגדרה כלומר ל-`this` של המטודה שהוא ה-`this` של האובייקט שה-`x` שלו הוא 42

5) Arrow vs function: In which situations should you ****not**** use an arrow function? Give two examples (e.g., constructors, methods needing dynamic `this`).

5. לפונקציית `arrow` אין `this` משלה הוא תמיד מתייחס לסביבה בה היא הוכרזה. כשמשתמשים במטודת `constructor` ה-`this` מתייחס למופע של האובייקט שבו נמצאת ה-`constructor` ולכן צריך פונקצייה רגילה שיש לה `this`. אותו דבר כשרוצים להגדיר פונקצייה שתפעל ע"י קולבק של `event` עשאנחנו רוצים שהפונקצייה תשנה משהו בהתאם לאלמנט שקרא לה צריך פונקצייה רגילה שהתייחסות ל-`this` בה יכולה להשתנות בהתאם למי שקרא לה. בפונקציית `arrow` ה-`this` מתייחס תמיד לאותו הדבר לכן היא לא מתאימה.

6) Default parameters: Explain the output.

```
function greet(name = 'Guest', exclam = name.length > 3) {  
  return `Hello, ${name}${exclam ? '!' : '.'}`;  
}  
console.log(greet());  
console.log(greet('Ana'));  
console.log(greet('Dima', false));
```

6. לפונקצייה שני פרמטרים `name` שהיא מדפיסה את הארגומנט שנשלח אליה אחרי `hello` לפרמטר הזה נקבע דיפולט `Guest` הפרמטר השני בודק אם זה `true` או `false` שאורכו של `name` גדול מ-3 במקרה שזה `true` יודפס סימן קריאה אחרי `name` ובמקרה ש-`false` תודפס שם נקודה.

בהדפסה הראשונה יודפס `Hello Guest!` כי לא מוגדר `name` ואורכו של השם הדיפולטיבי (במקרה שלא מוגדר `name`) גדול מ-3 לכן יהיה סימן קריאה.

בהדפסה השנייה יודפס `Hello Ana`. יש נקודה בסוף כי אורכו של `Ana` לא גדול מ-3

בהדפסה השלישית יודפס `Hello Dima` כי אמנם אורכו של `Dima` גדול מ-3 ו- צריך להיות `true` אבל שלחנו ארגומנט לפרמטר `exclam` שקובע שהוא `false`

ES6 (ES2015)

7) Rest parameters: Implement `sum( .nums)` returning the total. Why is `arguments` not ideal here?

// your code

```
389 function sum(...rest) {
390     const arr = [...rest].reduce((total, num)=>total = total + num);
391     console.log(arr);
392 }
393 sum(1,2,3,4,5,6)
```

21

[doogmaot.js:391](#)

7. זה לא יעיל לשלוח כל ארגומנט בנפרד בטח כשהמטרה היא לפעול על כל הארגומנטים כקבוצה ובטח אם במקרים שבהם לא נדע כמה מספרים יהיו בקבוצה

8) Spread operator: Convert `const set = new Set([1,2,3])` to an array and append `4` without mutating the original Set.

// your code

```
398 const set = new Set([1,2,3])
399
400 const setAsArrWith4 = [...set, 4]
401 console.log(set);
402 console.log(setAsArrWith4);
```

▶ Set(3) {1, 2, 3}

[doogmaot.js:401](#)

▶ (4) [1, 2, 3, 4]

[doogmaot.js:402](#)

9) Object literal enhancements: Rewrite to use shorthand properties, computed property names, and concise methods.

```
const x = 1, y = 2;
const key = 'sum';
const obj = {
  x: x,
  y: y,
  sum: function(){ return x + y; }
};
```

```
407 const obj = {
408     x: 1,
409     y: 2,
410     sum(){return this.x + this.y}
411 }
412 console.log(obj.sum());
```

3

[doogmaot.js:412](#)

ES6 (ES2015)

10) Destructuring arrays: Swap two variables without a temp and extract the first item and the rest.

```
let a = 10, b = 20;  
// swap here  
const arr = [1,2,3,4];  
// first = ?, rest = ?
```

```
417 let a = 10, b = 20;  
418 [a,b] = [b,a] // swap here  
419  
420 const arr = [1,2,3,4];  
421 const first = arr.splice(0,1); const rest = arr // first = ?, rest = ?  
422  
423 console.log("a="+a);  
424 console.log("b="+b);  
425 console.log('first: ' + first);  
426 console.log('rest: ' + rest);
```

a=20	doogmaot.js:423
b=10	doogmaot.js:424
first: 1	doogmaot.js:425
rest: 2,3,4	doogmaot.js:426

11) Destructuring objects: Given

```
const user = { id: 7, name: 'Avi', address: { city: 'TLV', zip: 12345 } };
```

Extract `id`, rename `name` to `fullName`, and get `city` with safe default `N/A` if `address` is missing.

```
431 const user = { id: 7, name: 'Avi', address: { city: 'TLV', zip: 12345 } };  
432  
433 let {id, name: fullName, address = {city: 'N/A'}} = user
```

ES6 (ES2015)

12) Template literals: Build a multi-line string with interpolation and a backtick character inside it. Show how to escape backticks.

```
438 const interpolation = 'backtick'
439 const multiLineString = `
440 This is a multi-line string
441 with interpolation
442 and ${interpolation} character inside it.
443 The ${interpolation} is this
444 very small thing here --> \` <--
445 By the way every where it reads:
446 "${interpolation}"
447 I inserted it using interpolation
448 `
449 console.log(multiLineString);
```

[doogmaot.js:449](#)

```
This is a multi-line string
with interpolation
and backtick character inside it.
The backtick is this
very small thing here --> ` <--
By the way every where it reads:
"backtick"
I inserted it using interpolation
```

13) Tagged templates: Write a simple tag function `safe` that HTML-escapes interpolations to avoid XSS.

```
const name = '<img onerror=alert(1) />';
console.log( safe`Hello, ${name}!` );
```

```
454 function safe(text) {
455     text = text
456     .replace(/</g, '&lt;');
457     .replace(/>/g, '&gt;');
458     .replace(/\\/g, '&#x2F;');
459     return text
460 }
461 const name = '<img onerror=alert(1) />';
462 console.log( safe`Hello, ${name}!` );
```

Hello, !

[doogmaot.js:462](#)

ES6 (ES2015)

14) Classes: Convert the following constructor/prototype code to an ES6 class with a getter.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.say = function(){ return 'Hi ' + this.name; };  
Object.defineProperty(Person.prototype, 'initial', {  
  get() { return this.name[0]; }  
});
```

```
466  class Person {  
467    constructor(name) {  
468      this.name = name  
469    }  
470    say()=>{  
471      return 'Hi ' + this.name  
472    }  
473  
474    get initial() {  
475      return this.name[0]  
476    }  
477  }  
478  
479  const guy = new Person('Guy')  
480  console.log(guy.say());  
481  console.log(guy.initial);
```

Hi Guy

[doogmaot.js:480](#)

G

[doogmaot.js:481](#)

ES6 (ES2015)

15) Inheritance and `super`: Implement `class Admin extends User` that calls `super(name)` and overrides `say()` while still using the parent logic.

```
486 class User {
487   constructor(name) {
488     this.name = name
489   }
490   say(){
491     console.log(`Hi, I am ${this.name}`);
492   }
493 }
494
495 class Admin extends User {
496   constructor(name) {
497     super (name)
498     this.role = 'Administrator'
499   }
500   say()=>{
501     super.say()
502     console.log(`${this.role}, this is my role here`);
503   }
504 }
505
506 const user = new User('David')
507 const admin = new Admin('Adel')
508 user.say()
509 console.log('-----')
510 admin.say()
```

Hi, I am David	doogmaot.js:491
-----	doogmaot.js:509
Hi, I am Adel	doogmaot.js:491
Administrator, this is my role here	doogmaot.js:502

ES6 (ES2015)

16) Static methods/fields: Create a class `Id` with a static counter and a static `next()` that returns incrementing IDs starting from 1.

```
517 class Id {
518     static counter = 0
519     constructor() {
520         this.id = Id.counter
521     }
522     static next(){
523         Id.counter++
524         return Id.counter
525     }
526 }
527
528
529 console.log(`The next ID is: ${Id.next()}`);
530 console.log(`The next ID is: ${Id.next()}`);
531 console.log(`The next ID is: ${Id.next()}`);
```

The next ID is: 1

[doogmaot.js:529](#)

The next ID is: 2

[doogmaot.js:530](#)

The next ID is: 3

[doogmaot.js:531](#)

17) Modules (named vs default): Given two files, show how to export a default and a named function from `math.js` and import them in `app.js`.

17. בקובץ ה- htmlk משתמשים ב-script מסוג module

```
9 <script type="module" src="./app.js"></script>
```

בקובץ math.js הפונקציה some מיוצאת כ- named והפונקציה msg כ- default

```
1 export const some = (a,b) => console.log(a + b);
2 const msg = () => console.log('The function is from math.js');
3 export default msg
```

בקובץ app.js הפונקציה some מיובאת כ- named והפונקציה msg כ- default

```
1 import msg, { some } from "./math.js";
```


ES6 (ES2015)

18) Re-exports: In `index.js`, re-export everything from `utils.js` and also export a default from `main.js` under the same module.

```
1 export * from './utils.js'
2 export {default} from './main.js'
```

19) Iterables: Implement a custom iterable `range(3,7)` that works with `for .of` and the spread operator.

```
// for (const n of range(3,7)) console.log(n); // 3,4,5,6,7
// [...range(1,3)] // [1,2,3]
```

```
562 function range(firstNo, lastNo) {
563     return {
564         [Symbol.iterator]() {
565             let currentNo = firstNo
566             return {
567                 next(){
568                     return currentNo <= lastNo ? {done: false, value: currentNo++} : {done: true}
569                 }
570             }
571         }
572     }
573 }
574
575 for(const n of range(3,7)){
576     console.log(n);
577 }
578 console.log([...range(1,3)]);
```

3	doogmaot.js:576
4	doogmaot.js:576
5	doogmaot.js:576
6	doogmaot.js:576
7	doogmaot.js:576
▶ (3) [1, 2, 3]	doogmaot.js:578

ES6 (ES2015)

20) Iterators vs for .in: Explain the difference between `for .of` and `for .in` with arrays and objects. Provide one example where each is appropriate.

```
583 const obj = {
584   citizen01: {name: 'Patrick', job: 'unemployed' },
585   citizen02: {name: 'SpongeBob', job: 'a frying cook' },
586   citizen03: {name: 'Sandy', job: 'a scientist' }
587 }
588 const arr = [
589   {name: 'Patrick', job: 'unemployed' },
590   {name: 'SpongeBob', job: 'a frying cook' },
591   {name: 'Sandy', job: 'a scientist' }
592 ]
593
594 for(const item of arr){
595   console.log(`${item.name} is ${item.job}`);
596 }
597 for(const key in obj){
598   console.log(`${obj[key].name} is ${obj[key].job}`);
599 }
600 }
```

Patrick is unemployed	doogmaot.js:595
SpongeBob is a frying cook	doogmaot.js:595
Sandy is a scientist	doogmaot.js:595
Patrick is unemployed	doogmaot.js:598
SpongeBob is a frying cook	doogmaot.js:598
Sandy is a scientist	doogmaot.js:598

21) Generators: Write a generator `powers(base, limit)` that yields `base^0` up to `base^limit`.

```
606 function* powers(base, limit){
607   for(let powerOf = 0; powerOf <= limit; powerOf++){
608     yield base**powerOf
609   }
610 }
611
612 console.log([...powers(2,3)]);
```

► (4) [1, 2, 4, 8]

[doogmaot.js:612](#)

ES6 (ES2015)

22) Map and Set: When would you prefer `Map` over a plain object, and `Set` over an array? Give concrete examples and complexity notes.

```
616 const user1 = 'Sandy'
617 const user2 = 'Patrick'
618
619 const map = new Map([
620   [user1, {team: 'k', rank: 2}],
621   [user2, {team: 'k', rank: 5}]
622 ])
623
624 const obj = {
625   user1: {team: 'k', rank: 2},
626   user2: {team: 'k', rank: 5}
627 }
628
629 console.log(map.get(user1));
630 console.log(obj.user1);
631
632 console.log(map.get('Sandy'));
633 console.log(obj['Sandy']);
634
635 map.delete('Sandy')
636 console.log(map);
637
638 delete obj.user1
639 console.log(obj);
640
```

▶ {team: 'k', rank: 2}	doogmaot.js:629
▶ {team: 'k', rank: 2}	doogmaot.js:630
▶ {team: 'k', rank: 2}	doogmaot.js:632
undefined	doogmaot.js:633
▶ Map(1) {'Patrick' => {...}}	doogmaot.js:636
▼ {user2: {...}} ⓘ	doogmaot.js:639
▶ user2: {team: 'k', rank: 5}	
▶ [[Prototype]]: Object	
1	doogmaot.js:641

ES6 (ES2015)

22. בקוד יש אובייקטים זהים של שני משתמשים פעם כ-Map ופעם כאובייקט רגיל. כמו כן, יצרתי שני קבועים עם שמות המשתמשים. האובייקט הרגיל יכול לקבל רק string כ- key. אבל Map יודע לקבל כל דבר גם קבועים ובזה השתמשתי.

בקוד ובתוצאותיו ניתן לראות את היתרון של שימוש ב-Map על שימוש באובייקט רגיל במקרה הזה.

כשרוצים להציג את user1 בשתי השיטות אפשר להשיג אותו על-פי ה-key (שורות 629, 630) אבל עם Map אפשר להשיג אותו גם בקריאה לשם המשתמש במקרה הזה Sandy (שורות 632, 633) בגלל שה-Map משתמש בקבוע כ- key הוא ידע לזהות את Sandy והאובייקט הרגיל שלא מכיר את Sandy יחזיר כמובן undefined. וזה ממש נוח.

למשל, אם נרצה למחוק את Sandy מה-Map נוכל פשוט לבקש למחוק את Sandy מהאובייקט הרגיל נצטרך לבקש למחוק את obj.user1 לך תזהה שזאת Sandy.

לבסוף, אם נרצה לדעת כמה משתמשים נותרו אחרי שמחקנו את Sandy ב-Map בגלל שהוא אינטרבל נוכל פשוט לבקש את ה-size (שזה כמו length במערך) ונראה שנותר 1 (שורה 641). אם נרצה לדעת את אותו נתון מהאובייקט הרגיל נצטרך לכתוב פונקציה כמו למשל בשורה 643 שתיתן לנו את התוצאה 1 (שורה 648)

```
641 console.log(map.size);
642
643 howMany=(objName)=>{
644     let count = 0
645     for(item in objName){
646         count++
647     }
648     console.log(count);
649 }
650
651 howMany(obj)
```

1 [doogmaot.js:641](#)

1 [doogmaot.js:648](#)

לגבי שימוש ב- Set על פני אובייקט רגיל, יש יתרון ברור לשימוש ב-Set כשרוצים להשתמש בעובדה שהוא מתעלם מערכים שמופיעים יותר מפעם במערך החדש הוא יוצר. בקוד להלן יש מערך answers שכולל תשובות של תלמידים לגבי נושאים שהם חושבים שיש לשפר. במקרה הזה המוסד החינוכי ששאל את השאלה לא מעוניין בכזה פעמים כל נושא לשיפור עלה אלא רק במה הם הנושאים שהתלמידים חושבים שיש לשפר. בקוד יצרתי Set על בסיס מערך התשובות והראתי שבסך הכל יש שני נושאים לשיפור (שורה 655).

```
653 const answers = ['homework', 'homework', 'homework', 'homework', 'class tasks', 'homework']
654 const whatDoStudentsWant = new Set (...answers)
655 console.log(whatDoStudentsWant);
```

► Set(2) {'homework', 'class tasks'}

[doogmaot.js:655](#)

ES6 (ES2015)

23) WeakMap & WeakSet: Explain their main difference from Map/Set and a real use case (e.g., caching, private fields).

23. ההבדל בין WeakMap/WeakSet ל-Map/Set הוא בכך שיש קשר חלש בין משתנים להתייחסויות אליהם בקוד. כלומר אם מוחקים משתנה ההתייחסות אליו תהפוך למועמדת למחיקה להלן קוד הסבר. בקוד להלן יש חישוב (שמייצג תהליך שגוזל משאבי זמן וזכרון) של משכורת של עובד. לאחר שחושבה היא מועברת ל-WeakMap בשם cache כשרוצים להשיג את המשכורת המחושבת אז אם היא כבר חושבה הקוד מביא אותה לפי שם העובד מ-cache ואם לא הוא מחשב אותה. כמו כן מודפס לקונסולה אם העובד נמצא ב-cache או לא. בתוצאות הקוד ניתן לראות איך זה פועל עם העובד Shimon בפעם הראשונה ובפעם השנייה. אחר כך מחושבת משכורת לעובדת Dana ואז שוב ל-Shimon שאנחנו רואים שהוא עדיין קיים ב-cache. אם נמחק את Shimon בגלל שהשתמשנו ב-WeakMap גם ההתייחסות אליו בחישוב המשכורת יהפוך למועמד למחיקה וימחק בסופו של דבר כשיגיע ה-Garbage collector. לו השתמשתי ב-Map היה נמחק אם לא היו מוחקים גם אותו ישירות והיה ממשיך לתפוס זכרון.

```
684 const cache = new WeakMap()
685
686 function payWorker(worker) {
687   if (cache.has(worker)) {
688     console.log('Worker found');
689     return cache.get(worker);
690   }
691   console.log('worker NOT found');
692   const howMuch = finalSalary(worker);
693   cache.set(worker, howMuch);
694
695   return howMuch;
696 }
697
698 function finalSalary(worker) {
699   return { ...worker, salary: worker.baseSalary*2+504-(1.08*(worker.baseSalary)) };
700 }
701
702 let worker1 = { name: "Shimon", baseSalary: 6900 };
703 let worker2 = { name: "Dana", baseSalary: 7200 };
704
705 console.log(payWorker(worker1));
706 console.log(payWorker(worker1));
707 console.log(payWorker(worker2));
708 console.log(payWorker(worker1));
```

worker NOT found	doogmaot.js:691
▶ {name: 'Shimon', baseSalary: 6900, salary: 6851.999999999999}	doogmaot.js:705
Worker found	doogmaot.js:688
▶ {name: 'Shimon', baseSalary: 6900, salary: 6851.999999999999}	doogmaot.js:706
worker NOT found	doogmaot.js:691
▶ {name: 'Dana', baseSalary: 7200, salary: 7127.999999999999}	doogmaot.js:707
Worker found	doogmaot.js:688
▶ {name: 'Shimon', baseSalary: 6900, salary: 6851.999999999999}	doogmaot.js:708

ES6 (ES2015)

להלן קוד הסבר לחלק השני של השאלה (אבטחה). ההסבר שלו בדף הבא.

```
711 const map = new Map()
712
713 class PersonUnSecure {
714   constructor(name) {
715     this.name = name
716     map.set(this, {secret: 'TOP SECRET'})
717   }
718 }
719
720 let person1 = new PersonUnSecure('Shimon')
721
722 console.log('Before nulling', person1.name, map.get(person1));
723
724 let whoUsed = [];
725 for (const x of map.entries()) {
726   whoUsed = [...x]
727 }
728 console.log(whoUsed);
729
730 person1 = null
731
732 console.log('After nulling', map.get(person1));
733
734 console.log(whoUsed);
735
736 console.log('-----');
737
738 const weakmap = new WeakMap()
739
740 class PersonSecure {
741   constructor(name) {
742     this.name = name
743     weakmap.set(this, {secret: 'TOP SECRET'})
744   }
745 }
746
747 let person2 = new PersonSecure('Dana')
748
749 console.log('Before nulling', person2.name, weakmap.get(person2));
750
751 person2 = null
752
753 console.log('After nulling', weakmap.get(person2));
```

Before nulling Shimon ▶ {secret: 'TOP SECRET'} [doogmaot.js:722](#)

▶ (2) [PersonUnSecure, {...}] [doogmaot.js:728](#)

After nulling undefined [doogmaot.js:732](#)

▼ (2) [PersonUnSecure, {...}] ⓘ [doogmaot.js:734](#)

▶ 0: PersonUnSecure {name: 'Shimon'}

▶ 1: {secret: 'TOP SECRET'}

length: 2

▶ [[Prototype]]: Array(0)

----- [doogmaot.js:736](#)

Before nulling Dana ▶ {secret: 'TOP SECRET'} [doogmaot.js:749](#)

After nulling undefined [doogmaot.js:753](#)

ES6 (ES2015)

23. (לגבי אבטחה) בקוד להלן אנחנו רואים שני Class לכל אחד מהם יש name ויש secret שהוא חלק סודי שנשמר ב-key שהוא קבוע המוגדר בקלאס PersonUnSecure Map- ובקלאס PersonSecure כ-WeakMap ב-PersonUnSecure גם אחרי שמוחקים את Shimon אחד המופעים שלו הסוד עדיין שמור ב-Map ואפשר להגיע אליו עם entriesf כמו שמודגם החל משורה 724 (תוצאה בשורה 734). הטענה היא (ואין לי הוכחה לכך) שב-WeakMap אחרי שמחקנו את Shimon בגלל הקשר החלש להתייחסויות בקוד הסוד ימחק כשה-Garbage collector יפעל וב-Map הוא ישאר אם לא נמחק אותו ישירות. אבל בכל מקרה ב-WeakMap אין שיטה שיכולה לקרוא את מה שיש כמו שעשיתי עם Map המטודה entries לא תעבוד כאן ולכן הסוד לא ייחשף.

24) Object methods: Use `Object.assign`, `Object.keys`, `Object.values`, `Object.entries` to clone an object and build a new one with transformed values.

24. יצרתי פונקציה *modify שעושה שינויים בארגומט שהיא מקבלת לפי ה-type שלו. הפונקציה משתמשת במטודת ה-entries של Object. בעקרון יכולתי להפעיל אותה ישירות על האובייקט originalObj והיא הייתה משכפלת ומשנה את הערכים בו. אבל לצורך הדגמת המטודות הנוספות שהוזכרו בשאלה השתמשתי בה רק כדי לשנות את אובייקט ה-Address שבתוכו.

לאחר מכן הדגמתי איך משכפלים ומשנים את ערכי האובייקט עם keys ו-values ולבסוף הדפסתי את האובייקט המשוכפל והמשונה ע"י שכפולו שוב הפעם באמצעות assign ובכך השתמשתי בכל ארבע המטודות כדי לשכפל ולשנות את ערכיו של originalObj.

*את הקוד של הפונקציה modify אפשר למצוא בעמוד הבא.

```
845   const originalObj = {
846     "First Name": "Adam",
847     "Last name": "King",
848     "Address": {
849       "Street name": "Baelin",
850       "Street number": 603
851     }
852   }
853   console.log('Original object:', originalObj);
854
855   const keys = Object.keys(originalObj)
856   const values = Object.values(originalObj)
857   let mObj2 = {}
858   for(const key in keys){
859     mObj2[keys[key]] = modify(values[key])
860   }
861   console.log('Cloned & modified:', Object.assign({}, mObj2));
```


ES6 (ES2015)

24. התוצאה של הקוד:

Original object:  {First Name: 'Adam', Last name: 'King', Address: {...}}  [doogmaot.js:853](#)
 ▶ Address: {Street name: 'Baelin', Street number: 603}
 First Name: "Adam"
 Last name: "King"
 ▶ [[Prototype]]: Object

Cloned & modified:  {First Name: 'A', Last name: 'K', Address: {...}}  [doogmaot.js:861](#)
 ▶ Address: {Street name: 'B', Street number: 22311}
 First Name: "A"
 Last name: "K"
 ▶ [[Prototype]]: Object

24. הקוד של הפונקציה modify שיצרתי

```
823 function modify(value) {  
824     let type = Object.prototype.toString.call(value)  
825     switch (type) {  
826         case '[object String]':  
827             return value = value[0]  
828         case '[object Number]':  
829             return value = value * 37  
830         case '[object Object]':  
831             let mObj = {}  
832             const entries = Object.entries(value)  
833             entries.forEach(prop => {  
834                 mObj[prop[0]] = modify(prop[1])  
835             })  
836             return mObj  
837         default:  
838             return value  
839     }  
840 }  
841  
842  
843  
844
```


ES6 (ES2015)

25) Array helpers added around ES6: Show examples of `Array.from`, `Array.of`, `find`, `findIndex`, `includes`. Explain a pitfall with `new Array(3)` vs `Array.of(3)`

```
878 const string = 'string'
879 const arrFromString = Array.from(string)
880 console.log(arrFromString);
```

▶ (6) ['s', 't', 'r', 'i', 'n', 'g']

[doogmaot.js:880](#)

```
884 const makeArrayOf = Array.of('k', 11, [1,2,3], 12)
885 console.log(makeArrayOf);
```

▼ (4) ['k', 11, Array(3), 12] ⓘ

[doogmaot.js:885](#)

0: "k"

1: 11

▶ 2: (3) [1, 2, 3]

3: 12

length: 4

▶ [[Prototype]]: Array(0)

```
889 const numbers = [57, 68, 100]
890
891 const largerThen60 = numbers.find(x => x > 60)
892 const smallerthan50 = numbers.find(x => x < 50)
893
894 console.log(largerThen60);
895 console.log(smallerthan50);
896
897 const largerThen60Index = numbers.findIndex(x => x > 60)
898 const smallerthan50Index = numbers.findIndex(x => x < 50)
899
900 console.log(largerThen60Index);
901 console.log(smallerthan50Index);
902
903 const has68 = numbers.includes(68)
904 const has50 = numbers.includes(50)
905
906 console.log(has68);
907 console.log(has50);
```

68	doogmaot.js:894
undefined	doogmaot.js:895
1	doogmaot.js:900
-1	doogmaot.js:901
true	doogmaot.js:906
false	doogmaot.js:907

ES6 (ES2015)

25. חשוב לא להתבלבל `new Array(3)` יוצר מערך חדש עם שלושה מקומות ריקים לעומתו `Array.of(3)` יוצר מערך חדש ובו אלמנט אחד - 3. להלן קוד להמחשה.

```
909 const arrX3 = new Array(3)
910 const the3Arr = Array.of(3)
911 console.log('new Array(3):', arrX3);
912 console.log('Array.of(3):', the3Arr);
```

`new Array(3):` ▶ (3) [empty × 3]

[doogmaot.js:911](#)

`Array.of(3):` ▶ [3]

[doogmaot.js:912](#)

26) Symbols: Create a symbol-keyed property on an object that does not show up in `for...in` or `Object.keys`, but is retrievable through `Object.getOwnPropertySymbols`.

```
918 const secretKey = Symbol('secret-key')
919 const objWithSecret = {
920   |   userName: 'Adam'
921   | }
922 objWithSecret[secretKey] = `Adam's secret`
923
924 console.log('objWithSecret[secretKey]:', objWithSecret[secretKey]);
925
926 console.log(Object.keys(objWithSecret));
927
928 for(key in objWithSecret){
929   |   console.log(key);
930   | }
931
932 console.log(Object.getOwnPropertySymbols(objWithSecret));
933 console.log(objWithSecret['secret-key']);
934 console.log(objWithSecret[Symbol('secret-key')]);
```

`objWithSecret[secretKey]:` Adam's secret

[doogmaot.js:924](#)

▶ ['userName']

[doogmaot.js:926](#)

`userName`

[doogmaot.js:929](#)

▶ [Symbol(secret-key)]

[doogmaot.js:932](#)

`undefined`

[doogmaot.js:933](#)

`undefined`

[doogmaot.js:934](#)

ES6 (ES2015)

27) Well-known symbols: Use `Symbol.iterator` to make a plain object iterable. Optionally, demonstrate `Symbol.toStringTag` to customize `Object.prototype.toString.call(obj)`.

```
941 const specialObject = {
942   items: ['Banana', 'Apple', ' Pear'],
943   [Symbol.iterator]() {
944     let index = 0;
945     const self = this;
946     return {
947       next() {
948         return index < self.items.length ? { value: self.items[index++], done: false } : { value: undefined, done: true }
949       }
950     };
951   },
952   [Symbol.toStringTag]: 'fruits'
953 };
954
955 for(item of specialObject){
956   console.log(item);
957 }
958 console.log('the type of specialObject is:', Object.prototype.toString.call(specialObject));
```

Banana	doogmaot.js:956
Apple	doogmaot.js:956
Pear	doogmaot.js:956
the type of specialObject is: [object fruits]	doogmaot.js:958

ES6 (ES2015)

28) Promises: Convert a callback-style `readFile(path, cb)` into a `readFileP(path)` that returns a Promise. Show success and error handling with `.then/.catch`.

```
988  function readFileP(path){
989      return new Promise((resolve, reject) => {
990          readFile(path, (fileData, err)=>{
991              if(fileData){
992                  resolve(fileData)
993              }else{
994                  reject(err)
995              }
996          })
997      })
998  }
999  readFileP('../file.pdf')
1000  .then((pathData) =>{
1001      console.log(pathData);
1002  })
1003  .catch((err) => {
1004      console.log(err);
1005  })
```

ES6 (ES2015)

29) Promise chaining & microtasks: What is the output order?

```
console.log('A');  
Promise.resolve().then(() => console.log('B'));  
console.log('C');  
queueMicrotask(() => console.log('D'));
```

29. קודם יודפס A ומיד אחריו C שני פלטים של פעולה סינכרונית מיד אחר כך הפעולות הא-סינכרוניות שנכנסו לתור לפי סדר הכנסתן ידפיסו קודם B ואחר כך D

```
1000 console.log('A');  
1001 Promise.resolve().then(() => console.log('B'));  
1002 console.log('C');  
1003 queueMicrotask(() => console.log('D'));
```

A	doogmaot.js:1000
C	doogmaot.js:1002
B	doogmaot.js:1001
D	doogmaot.js:1003

30) Promise utilities: Implement a simplified `all(promises)` that resolves to an array of results in order, or rejects immediately on the first rejection.

```
1050 function all(promises) {  
1051     return new Promise((resolve, reject)=>{  
1052         const resArr = []  
1053         let resCount = 0  
1054  
1055         if(promises.length === 0){  
1056             resolve([])  
1057             return  
1058         }  
1059  
1060         promises.forEach((promise, i) => {  
1061             promise.then(value =>{  
1062                 resArr[i] = value  
1063                 resCount++  
1064  
1065                 if(resCount === promises.length){  
1066                     resolve(resArr)  
1067                 }  
1068             })  
1069             .catch(reason => reject(reason))  
1070         });  
1071     });  
1072 }
```